

Chương 8 – Giảm chiều dữ liệu

Cuốn sổ tay này chứa tất cả mã mẫu và giải pháp cho các bài tập trong chương 8.

 [Open in Colab](#)  [Open in Kaggle](#)

▼ Thiết lập

Dự án này yêu cầu Python 3.7 trở lên.

```
import sys

assert sys.version_info >= (3, 7)
```

Nó cũng yêu cầu Scikit-Learn $\geq 1.0.1$:

```
from packaging import version
import sklearn

assert version.parse(sklearn.__version__) >= version.parse("1.0.1")
```

Như chúng ta đã thực hiện trong các chương trước, hãy định nghĩa các kích thước phông chữ mặc định để làm cho các hình ảnh trở nên đẹp hơn:

```
import matplotlib.pyplot as plt

plt.rc('font', size=14)
plt.rc('axes', labelsz=14, titlesz=14)
plt.rc('legend', fontsize=14)
plt.rc('xtick', labelsz=10)
plt.rc('ytick', labelsz=10)
```

Và hãy tạo thư mục `images/dim_reduction` (nếu nó chưa tồn tại), và định nghĩa hàm `save_fig()` được sử dụng trong toàn bộ sổ tay này để lưu các hình ảnh với độ phân giải cao cho cuốn sách:

```
from pathlib import Path

IMAGES_PATH = Path() / "images" / "dim_reduction"
IMAGES_PATH.mkdir(parents=True, exist_ok=True)

def save_fig(fig_id, tight_layout=True, fig_extension="png", resolution=300):
    path = IMAGES_PATH / f"{fig_id}.{fig_extension}"
    if tight_layout:
        plt.tight_layout()
    plt.savefig(path, format=fig_extension, dpi=resolution)
```

▼ PCA

Chương này bắt đầu bằng một số hình ảnh để giải thích các khái niệm về PCA và N học Đường cong. Dưới đây là mã để tạo ra những hình ảnh này. Bạn có thể bỏ qua trực tiếp phần Thành phần chính bên dưới nếu bạn muốn.

Hãy tạo ra một tập dữ liệu 3D nhỏ. Nó có hình oval, xoay quanh không gian 3D, với các điểm phân bố không đều, và có khá nhiều tiếng ồn:

```
# extra code

import numpy as np
from scipy.spatial.transform import Rotation

m = 60
```

```

X = np.zeros((m, 3)) # initialize 3D dataset
np.random.seed(42)
angles = (np.random.rand(m) ** 3 + 0.5) * 2 * np.pi # uneven distribution
X[:, 0], X[:, 1] = np.cos(angles), np.sin(angles) * 0.5 # oval
X += 0.28 * np.random.randn(m, 3) # add more noise
X = Rotation.from_rotvec([np.pi / 29, -np.pi / 20, np.pi / 4]).apply(X)
X += [0.2, 0, 0.2] # shift a bit

```

Vẽ tập dữ liệu 3D, với mặt phẳng chiếu.

```

# extra code - this cell generates and saves Figure 8-2

import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X2D = pca.fit_transform(X) # dataset reduced to 2D
X3D_inv = pca.inverse_transform(X2D) # 3D position of the projected samples
X_centered = X - X.mean(axis=0)
U, s, Vt = np.linalg.svd(X_centered)

axes = [-1.4, 1.4, -1.4, 1.4, -1.1, 1.1]
x1, x2 = np.meshgrid(np.linspace(axes[0], axes[1], 10),
                     np.linspace(axes[2], axes[3], 10))
w1, w2 = np.linalg.solve(Vt[:2, :2], Vt[:2, 2]) # projection plane coeffs
z = w1 * (x1 - pca.mean_[0]) + w2 * (x2 - pca.mean_[1]) - pca.mean_[2] # plane
X3D_above = X[X[:, 2] >= X3D_inv[:, 2]] # samples above plane
X3D_below = X[X[:, 2] < X3D_inv[:, 2]] # samples below plane

fig = plt.figure(figsize=(9, 9))
ax = fig.add_subplot(111, projection="3d")

# plot samples and projection lines below plane first
ax.plot(X3D_below[:, 0], X3D_below[:, 1], X3D_below[:, 2], "ro", alpha=0.3)
for i in range(m):
    if X[i, 2] < X3D_inv[i, 2]:
        ax.plot([X[i][0], X3D_inv[i][0]],
                [X[i][1], X3D_inv[i][1]],
                [X[i][2], X3D_inv[i][2]], ":", color="#F88")

ax.plot_surface(x1, x2, z, alpha=0.1, color="b") # projection plane
ax.plot(X3D_inv[:, 0], X3D_inv[:, 1], X3D_inv[:, 2], "b+") # projected samples
ax.plot(X3D_inv[:, 0], X3D_inv[:, 1], X3D_inv[:, 2], "b.")

# now plot projection lines and samples above plane
for i in range(m):
    if X[i, 2] >= X3D_inv[i, 2]:
        ax.plot([X[i][0], X3D_inv[i][0]],
                [X[i][1], X3D_inv[i][1]],
                [X[i][2], X3D_inv[i][2]], "r--")

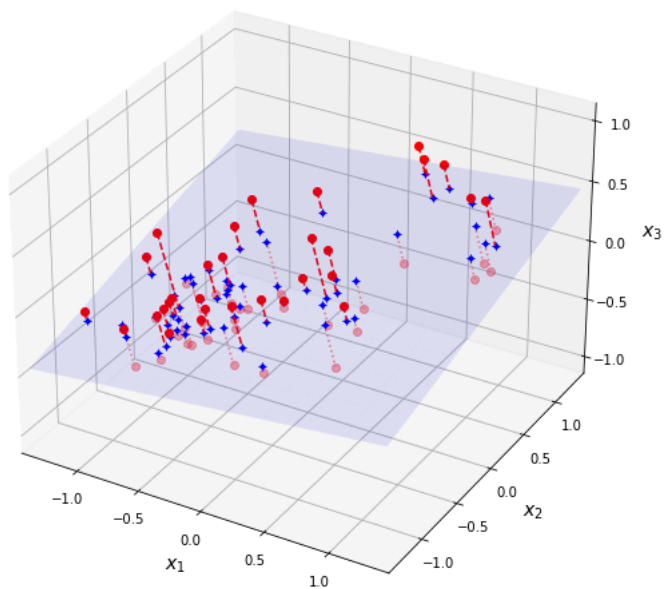
ax.plot(X3D_above[:, 0], X3D_above[:, 1], X3D_above[:, 2], "ro")

def set_xyz_axes(ax, axes):
    ax.xaxis.set_rotate_label(False)
    ax.yaxis.set_rotate_label(False)
    ax.zaxis.set_rotate_label(False)
    ax.set_xlabel("$x_1$", labelpad=8, rotation=0)
    ax.set_ylabel("$x_2$", labelpad=8, rotation=0)
    ax.set_zlabel("$x_3$", labelpad=8, rotation=0)
    ax.set_xlim(axes[0:2])
    ax.set_ylim(axes[2:4])
    ax.set_zlim(axes[4:6])

set_xyz_axes(ax, axes)
ax.set_zticks([-1, -0.5, 0, 0.5, 1])

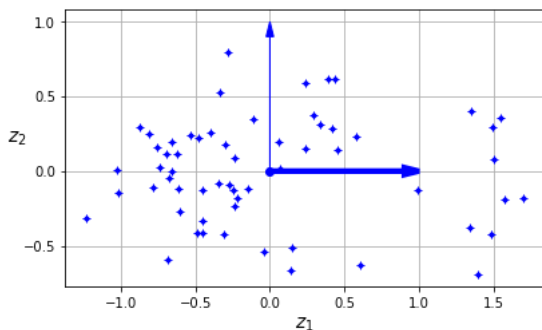
save_fig("dataset_3d_plot", tight_layout=False)
plt.show()

```



extra code - this cell generates and saves Figure 8-3

```
fig = plt.figure()
ax = fig.add_subplot(1, 1, 1, aspect='equal')
ax.plot(X2D[:, 0], X2D[:, 1], "b+")
ax.plot(X2D[:, 0], X2D[:, 1], "b.")
ax.plot([0], [0], "bo")
ax.arrow(0, 0, 1, 0, head_width=0.05, length_includes_head=True,
        head_length=0.1, fc='b', ec='b', linewidth=4)
ax.arrow(0, 0, 0, 1, head_width=0.05, length_includes_head=True,
        head_length=0.1, fc='b', ec='b', linewidth=1)
ax.set_xlabel("$z_1$")
ax.set_yticks([-0.5, 0, 0.5, 1])
ax.set_ylabel("$z_2$", rotation=0)
ax.set_axisbelow(True)
ax.grid(True)
save_fig("dataset_2d_plot")
```



```
from sklearn.datasets import make_swiss_roll
```

```
X_swiss, t = make_swiss_roll(n_samples=1000, noise=0.2, random_state=42)
```

extra code - this cell generates and saves Figure 8-4

```
from matplotlib.colors import ListedColormap
```

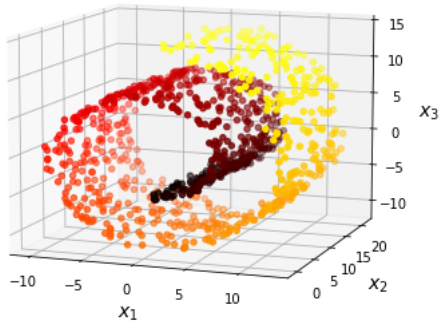
```
darker_hot = ListedColormap(plt.cm.hot(np.linspace(0, 0.8, 256)))
```

```
axes = [-11.5, 14, -2, 23, -12, 15]
```

```
fig = plt.figure(figsize=(6, 5))
```

```
ax = fig.add_subplot(111, projection='3d')
```

```
ax.scatter(X_swiss[:, 0], X_swiss[:, 1], X_swiss[:, 2], c=t, cmap=darker_hot)
ax.view_init(10, -70)
set_xyz_axes(ax, axes)
save_fig("swiss_roll_plot")
plt.show()
```



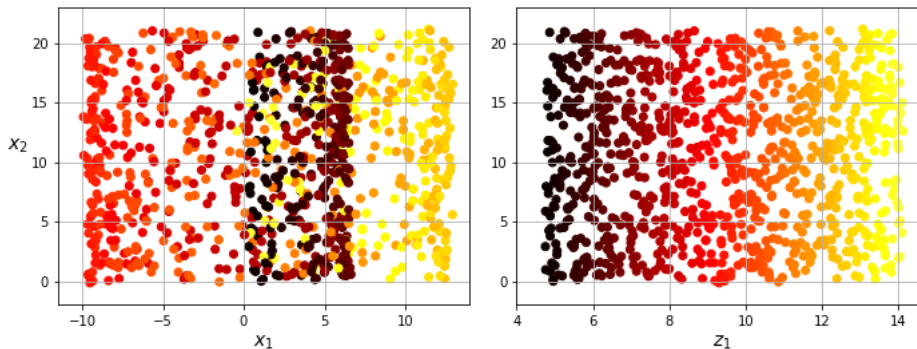
extra code - this cell generates and saves plots for Figure 8-5

```
plt.figure(figsize=(10, 4))

plt.subplot(121)
plt.scatter(X_swiss[:, 0], X_swiss[:, 1], c=t, cmap=darker_hot)
plt.axis(axes[:4])
plt.xlabel("$x_1$")
plt.ylabel("$x_2$", labelpad=10, rotation=0)
plt.grid(True)

plt.subplot(122)
plt.scatter(t, X_swiss[:, 1], c=t, cmap=darker_hot)
plt.axis([4, 14.8, axes[2], axes[3]])
plt.xlabel("$z_1$")
plt.grid(True)

save_fig("squished_swiss_roll_plot")
plt.show()
```



extra code - this cell generates and saves plots for Figure 8-6

```
axes = [-11.5, 14, -2, 23, -12, 15]
x2s = np.linspace(axes[2], axes[3], 10)
x3s = np.linspace(axes[4], axes[5], 10)
x2, x3 = np.meshgrid(x2s, x3s)

positive_class = X_swiss[:, 0] > 5
X_pos = X_swiss[positive_class]
X_neg = X_swiss[~positive_class]

fig = plt.figure(figsize=(6, 5))
ax = plt.subplot(1, 1, 1, projection='3d')
ax.view_init(10, -70)
ax.plot(X_neg[:, 0], X_neg[:, 1], X_neg[:, 2], "y^")
```

```

ax.plot_wireframe(5, x2, x3, alpha=0.5)
ax.plot(X_pos[:, 0], X_pos[:, 1], X_pos[:, 2], "gs")
set_xyz_axes(ax, axes)
save_fig("manifold_decision_boundary_plot1")
plt.show()

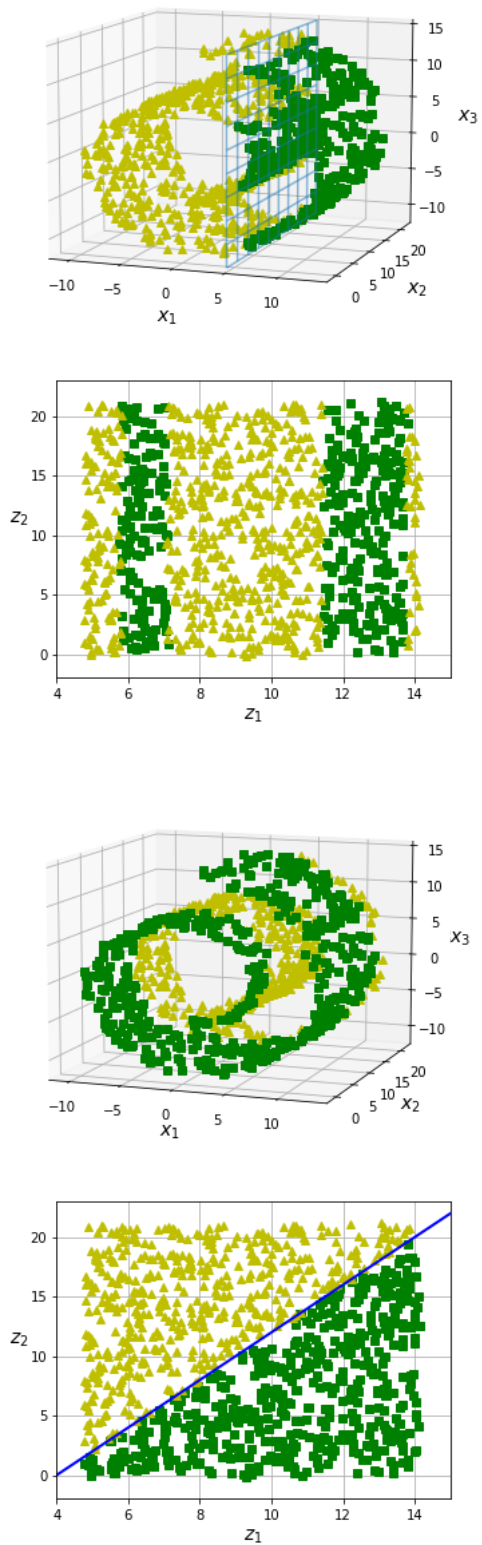
fig = plt.figure(figsize=(5, 4))
ax = plt.subplot(1, 1, 1)
ax.plot(t[positive_class], X_swiss[positive_class, 1], "gs")
ax.plot(t[~positive_class], X_swiss[~positive_class, 1], "y^")
ax.axis([4, 15, axes[2], axes[3]])
ax.set_xlabel("$z_1$")
ax.set_ylabel("$z_2$", rotation=0, labelpad=8)
ax.grid(True)
save_fig("manifold_decision_boundary_plot2")
plt.show()

positive_class = 2 * (t[:] - 4) > X_swiss[:, 1]
X_pos = X_swiss[positive_class]
X_neg = X_swiss[~positive_class]

fig = plt.figure(figsize=(6, 5))
ax = plt.subplot(1, 1, 1, projection='3d')
ax.view_init(10, -70)
ax.plot(X_neg[:, 0], X_neg[:, 1], X_neg[:, 2], "y^")
ax.plot(X_pos[:, 0], X_pos[:, 1], X_pos[:, 2], "gs")
ax.xaxis.set_rotate_label(False)
ax.yaxis.set_rotate_label(False)
ax.zaxis.set_rotate_label(False)
ax.set_xlabel("$x_1$", rotation=0)
ax.set_ylabel("$x_2$", rotation=0)
ax.set_zlabel("$x_3$", rotation=0)
ax.set_xlim(axes[0:2])
ax.set_ylim(axes[2:4])
ax.set_zlim(axes[4:6])
save_fig("manifold_decision_boundary_plot3")
plt.show()

fig = plt.figure(figsize=(5, 4))
ax = plt.subplot(1, 1, 1)
ax.plot(t[positive_class], X_swiss[positive_class, 1], "gs")
ax.plot(t[~positive_class], X_swiss[~positive_class, 1], "y^")
ax.plot([4, 15], [0, 22], "b-", linewidth=2)
ax.axis([4, 15, axes[2], axes[3]])
ax.set_xlabel("$z_1$")
ax.set_ylabel("$z_2$", rotation=0, labelpad=8)
ax.grid(True)
save_fig("manifold_decision_boundary_plot4")
plt.show()

```



extra code - this cell generates and saves Figure 8-7

```
angle = np.pi / 5
stretch = 5
m = 200

np.random.seed(3)
X_line = np.random.randn(m, 2) / 10
X_line = X_line @ np.array([[stretch, 0], [0, 1]]) # stretch
X_line = X_line @ [[np.cos(angle), np.sin(angle)],
```

```

[ np.sin(angle), np.cos(angle)]] # rotate

u1 = np.array([np.cos(angle), np.sin(angle)])
u2 = np.array([np.cos(angle - 2 * np.pi / 6), np.sin(angle - 2 * np.pi / 6)])
u3 = np.array([np.cos(angle - np.pi / 2), np.sin(angle - np.pi / 2)])

X_proj1 = X_line @ u1.reshape(-1, 1)
X_proj2 = X_line @ u2.reshape(-1, 1)
X_proj3 = X_line @ u3.reshape(-1, 1)

plt.figure(figsize=(8, 4))
plt.subplot2grid((3, 2), (0, 0), rowspan=3)
plt.plot([-1.4, 1.4], [-1.4 * u1[1] / u1[0], 1.4 * u1[1] / u1[0]], "k-",
         linewidth=2)
plt.plot([-1.4, 1.4], [-1.4 * u2[1] / u2[0], 1.4 * u2[1] / u2[0]], "k--",
         linewidth=2)
plt.plot([-1.4, 1.4], [-1.4 * u3[1] / u3[0], 1.4 * u3[1] / u3[0]], "k:",
         linewidth=2)
plt.plot(X_line[:, 0], X_line[:, 1], "ro", alpha=0.5)
plt.arrow(0, 0, u1[0], u1[1], head_width=0.1, linewidth=4, alpha=0.9,
         length_includes_head=True, head_length=0.1, fc="b", ec="b", zorder=10)
plt.arrow(0, 0, u3[0], u3[1], head_width=0.1, linewidth=1, alpha=0.9,
         length_includes_head=True, head_length=0.1, fc="b", ec="b", zorder=10)
plt.text(u1[0] + 0.1, u1[1] - 0.05, r"$\mathbf{c_1}$", color="blue")
plt.text(u3[0] + 0.1, u3[1], r"$\mathbf{c_2}$", color="blue")
plt.xlabel("$x_1$")
plt.ylabel("$x_2$", rotation=0)
plt.axis([-1.4, 1.4, -1.4, 1.4])
plt.grid()

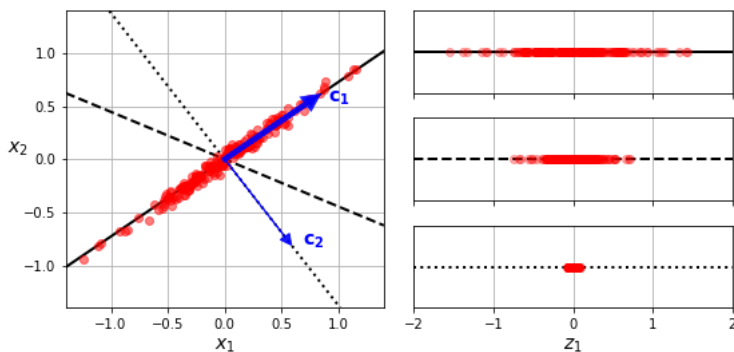
plt.subplot2grid((3, 2), (0, 1))
plt.plot([-2, 2], [0, 0], "k-", linewidth=2)
plt.plot(X_proj1[:, 0], np.zeros(m), "ro", alpha=0.3)
plt.gca().get_yaxis().set_ticks([])
plt.gca().get_xaxis().set_ticklabels([])
plt.axis([-2, 2, -1, 1])
plt.grid()

plt.subplot2grid((3, 2), (1, 1))
plt.plot([-2, 2], [0, 0], "k--", linewidth=2)
plt.plot(X_proj2[:, 0], np.zeros(m), "ro", alpha=0.3)
plt.gca().get_yaxis().set_ticks([])
plt.gca().get_xaxis().set_ticklabels([])
plt.axis([-2, 2, -1, 1])
plt.grid()

plt.subplot2grid((3, 2), (2, 1))
plt.plot([-2, 2], [0, 0], "k:", linewidth=2)
plt.plot(X_proj3[:, 0], np.zeros(m), "ro", alpha=0.3)
plt.gca().get_yaxis().set_ticks([])
plt.axis([-2, 2, -1, 1])
plt.xlabel("$z_1$")
plt.grid()

save_fig("pca_best_projection_plot")
plt.show()

```



- Các thành phần chính

```
import numpy as np

# X = [...] # the small 3D dataset was created earlier in this notebook
X_centered = X - X.mean(axis=0)
U, s, Vt = np.linalg.svd(X_centered)
c1 = Vt[0]
c2 = Vt[1]
```

Lưu ý: về nguyên tắc, thuật toán phân tích SVD (phân tích giá trị suy biến) trả về ba ma trận, $\mathbf{U}$, Σ và $\mathbf{V}$, sao cho $X = U\Sigma V^T$, trong đó $\mathbf{U}$ là một ma trận $m \times m$, Σ là một ma trận $m \times n$, và $\mathbf{V}$ là một ma trận $n \times n$. Nhưng hàm `svd()` lại trả về $\mathbf{U}$, $\mathbf{s}$ và V^T thay thế. $\mathbf{s}$ là vector chứa tất cả các giá trị trên đường chéo chính của n hàng đầu tiên của Σ . Vì Σ toàn là số không ở các vị trí khác, bạn có thể dễ dàng tái tạo nó từ $\mathbf{s}$, như sau:

```
# extra code - shows how to construct Σ from s
m, n = X.shape
Σ = np.zeros_like(X_centered)
Σ[:n, :n] = np.diag(s)
assert np.allclose(X_centered, U @ Σ @ Vt)
```

✓ Chiều xuống d kích thước

```
W2 = Vt[:2].T
X2D = X_centered @ W2
```

✓ Dùng Scikit-Learn

Với Scikit-Learn, PCA thực sự rất đơn giản. Nó thậm chí còn tự động xử lý việc trung tâm hóa trung bình cho bạn:

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X2D = pca.fit_transform(X)
```

```
pca.components_

array([[ 0.67857588,  0.70073508,  0.22023881],
       [ 0.72817329, -0.6811147 , -0.07646185]])
```

✓ Tỷ lệ phương sai giải thích

Bây giờ hãy xem xét tỷ lệ phương sai được giải thích:

```
pca.explained_variance_ratio_

array([0.7578477 , 0.15186921])
```

Chiều thứ nhất giải thích khoảng 76% sự biến thiên, trong khi chiều thứ hai giải thích khoảng 15%.

Khi giảm xuống 2D, chúng ta đã mất khoảng 9% sự biến thiên:

```
1 - pca.explained_variance_ratio_.sum() # extra code

0.09028309326742046
```

✓ Chọn Số Kích Thước Phù Hợp.

```
from sklearn.datasets import fetch_openml

mnist = fetch_openml('mnist_784', as_frame=False, parser="auto")
X_train, y_train = mnist.data[:60_000], mnist.target[:60_000]
X_test, y_test = mnist.data[60_000:], mnist.target[60_000:]
```



```
pca = PCA()
pca.fit(X_train)
cumsum = np.cumsum(pca.explained_variance_ratio_)
d = np.argmax(cumsum >= 0.95) + 1 # d equals 154
```

Lưu ý: Tôi đã thêm `parser="auto"` khi gọi `fetch_openml()` để tránh một cảnh báo về việc giá trị mặc định cho tham số đó sẽ thay đổi trong tương lai (điều này không liên quan trong trường hợp này). Vui lòng xem [documentation](#) để biết thêm chi tiết.

```
d
```

```
154
```

```
pca = PCA(n_components=0.95)
X_reduced = pca.fit_transform(X_train)
```

```
pca.n_components_
```

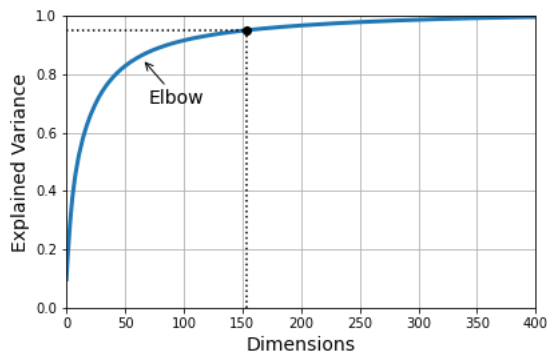
```
154
```

```
pca.explained_variance_ratio_.sum() # extra code
```

```
0.9501960192613035
```

```
# extra code - this cell generates and saves Figure 8-8
```

```
plt.figure(figsize=(6, 4))
plt.plot(cumsum, linewidth=3)
plt.axis([0, 400, 0, 1])
plt.xlabel("Dimensions")
plt.ylabel("Explained Variance")
plt.plot([d, d], [0, 0.95], "k:")
plt.plot([0, d], [0.95, 0.95], "k:")
plt.plot(d, 0.95, "ko")
plt.annotate("Elbow", xy=(65, 0.85), xytext=(70, 0.7),
            arrowprops=dict(arrowstyle="->"))
plt.grid(True)
save_fig("explained_variance_plot")
plt.show()
```



```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV
from sklearn.pipeline import make_pipeline

clf = make_pipeline(PCA(random_state=42),
                    RandomForestClassifier(random_state=42))
param_distrib = {
    "pca_n_components": np.arange(10, 80),
    "randomforestclassifier_n_estimators": np.arange(50, 500)
}
rnd_search = RandomizedSearchCV(clf, param_distrib, n_iter=10, cv=3,
                               random_state=42)
rnd_search.fit(X_train[:1000], y_train[:1000])

RandomizedSearchCV(cv=3,
                  estimator=Pipeline(steps=[('pca', PCA(random_state=42)),
                                             ('randomforestclassifier',
                                              RandomForestClassifier(random_state=42))]),
```

```

        param_distributions={'pca__n_components': array([10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24,
25, 26,
27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43,
44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60,
6...
414, 415, 416, 417, 418, 419, 420, 421, 422, 423, 424, 425, 426,
427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437, 438, 439,
440, 441, 442, 443, 444, 445, 446, 447, 448, 449, 450, 451, 452,
453, 454, 455, 456, 457, 458, 459, 460, 461, 462, 463, 464, 465,
466, 467, 468, 469, 470, 471, 472, 473, 474, 475, 476, 477, 478,
479, 480, 481, 482, 483, 484, 485, 486, 487, 488, 489, 490, 491,
492, 493, 494, 495, 496, 497, 498, 499])},
        random_state=42)

```

```
print(rnd_search.best_params_)
```

```
{'randomforestclassifier__n_estimators': 465, 'pca__n_components': 23}
```

```

from sklearn.linear_model import SGDClassifier
from sklearn.model_selection import GridSearchCV

```

```

clf = make_pipeline(PCA(random_state=42), SGDClassifier())
param_grid = {"pca__n_components": np.arange(10, 80)}
grid_search = GridSearchCV(clf, param_grid, cv=3)
grid_search.fit(X_train[:1000], y_train[:1000])

```

```

GridSearchCV(cv=3,
             estimator=Pipeline(steps=[('pca', PCA(random_state=42)),
                                       ('sgdclassifier', SGDClassifier())]),
             param_grid={'pca__n_components': array([10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26,
27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43,
44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60,
61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77,
78, 79])})

```

```
grid_search.best_params_
```

```
{'pca__n_components': 67}
```

▼ PCA cho Nén Dữ Liệu

```

pca = PCA(0.95)
X_reduced = pca.fit_transform(X_train, y_train)

```

```
X_recovered = pca.inverse_transform(X_reduced)
```

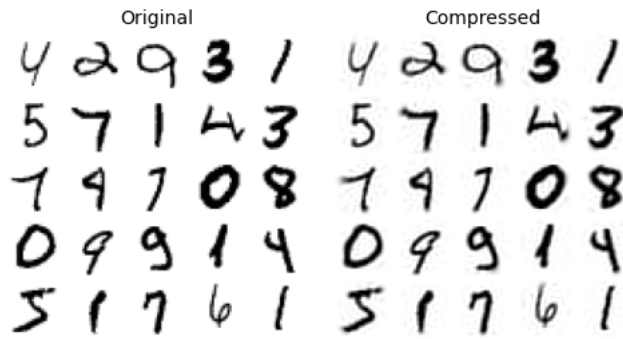
extra code - this cell generates and saves Figure 8-9

```

plt.figure(figsize=(7, 4))
for idx, X in enumerate((X_train[:2100], X_recovered[:2100])):
    plt.subplot(1, 2, idx + 1)
    plt.title(["Original", "Compressed"][idx])
    for row in range(5):
        for col in range(5):
            plt.imshow(X[row * 5 + col].reshape(28, 28), cmap="binary",
                      vmin=0, vmax=255, extent=(row, row + 1, col, col + 1))
            plt.axis([0, 5, 0, 5])
            plt.axis("off")

save_fig("mnist_compression_plot")

```



✓ PCA ngẫu nhiên

```
rnd_pca = PCA(n_components=154, svd_solver="randomized", random_state=42)
X_reduced = rnd_pca.fit_transform(X_train)
```

✓ PCA gia tăng

```
from sklearn.decomposition import IncrementalPCA

n_batches = 100
inc_pca = IncrementalPCA(n_components=154)
for X_batch in np.array_split(X_train, n_batches):
    inc_pca.partial_fit(X_batch)

X_reduced = inc_pca.transform(X_train)
```

Sử dụng lớp memmap của NumPy – một ánh xạ bộ nhớ tới một mảng được lưu trữ trong một tệp nhị phân trên đĩa.

Hãy tạo một thể hiện memmap, sao chép tập dữ liệu huấn luyện MNIST vào đó và gọi flush(), điều này đảm bảo rằng bất kỳ dữ liệu nào vẫn còn trong bộ nhớ đệm sẽ được lưu vào đĩa. Điều này thường được thực hiện bởi một chương trình đầu tiên:

```
filename = "my_mnist.mmap"
X_mmap = np.memmap(filename, dtype='float32', mode='write', shape=X_train.shape)
X_mmap[:] = X_train # could be a loop instead, saving the data chunk by chunk
X_mmap.flush()
```

Tiếp theo, một chương trình khác sẽ tải dữ liệu và sử dụng nó để đào tạo.

```
X_mmap = np.memmap(filename, dtype="float32", mode="readonly").reshape(-1, 784)
batch_size = X_mmap.shape[0] // n_batches
inc_pca = IncrementalPCA(n_components=154, batch_size=batch_size)
inc_pca.fit(X_mmap)

IncrementalPCA(batch_size=600, n_components=154)
```

✓ Chiếu ngẫu nhiên

Cảnh báo: phần này sẽ sử dụng khoảng 2.5 GB RAM. Nếu máy tính của bạn hết bộ nhớ, chỉ cần giảm m và n.

```
from sklearn.random_projection import johnson_lindenstrauss_min_dim

m, ε = 5_000, 0.1
d = johnson_lindenstrauss_min_dim(m, eps=ε)
d
```

7300

```
# extra code - show the equation computed by johnson_lindenstrauss_min_dim
d = int(4 * np.log(m) / (ε ** 2 / 2 - ε ** 3 / 3))
```

```
d
```

```
7300
```

```
n = 20_000
np.random.seed(42)
P = np.random.randn(d, n) / np.sqrt(d) # std dev = square root of variance

X = np.random.randn(m, n) # generate a fake dataset
X_reduced = X @ P.T
```

```
from sklearn.random_projection import GaussianRandomProjection

gaussian_rnd_proj = GaussianRandomProjection(eps=ε, random_state=42)
X_reduced = gaussian_rnd_proj.fit_transform(X) # same result as above
```

Cảnh báo, ô dưới đây có thể mất vài phút để chạy.

```
components_pinv = np.linalg.pinv(gaussian_rnd_proj.components_)
X_recovered = X_reduced @ components_pinv.T
```

```
# extra code - performance comparison between Gaussian and Sparse RP

from sklearn.random_projection import SparseRandomProjection

print("GaussianRandomProjection fit")
%timeit GaussianRandomProjection(random_state=42).fit(X)
print("SparseRandomProjection fit")
%timeit SparseRandomProjection(random_state=42).fit(X)

gaussian_rnd_proj = GaussianRandomProjection(random_state=42).fit(X)
sparse_rnd_proj = SparseRandomProjection(random_state=42).fit(X)
print("GaussianRandomProjection transform")
%timeit gaussian_rnd_proj.transform(X)
print("SparseRandomProjection transform")
%timeit sparse_rnd_proj.transform(X)
```

```
GaussianRandomProjection fit
4.05 s ± 327 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
SparseRandomProjection fit
3.85 s ± 647 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
GaussianRandomProjection transform
11.1 s ± 507 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
SparseRandomProjection transform
5.37 s ± 640 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

✓ LLE

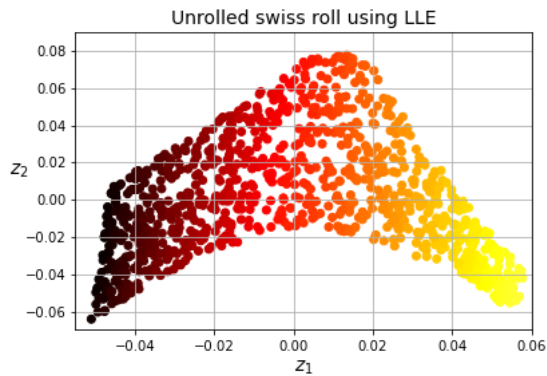
```
from sklearn.datasets import make_swiss_roll
from sklearn.manifold import LocallyLinearEmbedding

X_swiss, t = make_swiss_roll(n_samples=1000, noise=0.2, random_state=42)
lle = LocallyLinearEmbedding(n_components=2, n_neighbors=10, random_state=42)
X_unrolled = lle.fit_transform(X_swiss)
```

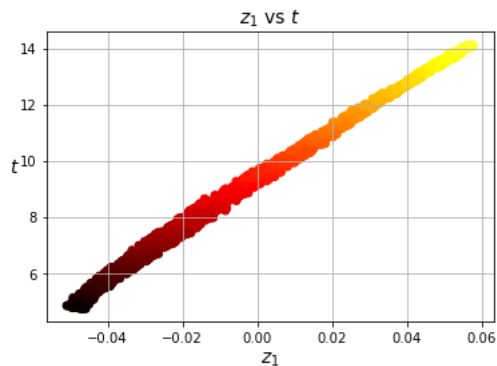
```
# extra code - this cell generates and saves Figure 8-10

plt.scatter(X_unrolled[:, 0], X_unrolled[:, 1],
            c=t, cmap=darker_hot)
plt.xlabel("$z_1$")
plt.ylabel("$z_2$", rotation=0)
plt.axis([-0.055, 0.060, -0.070, 0.090])
plt.grid(True)

save_fig("lle_unrolling_plot")
plt.title("Unrolled swiss roll using LLE")
plt.show()
```



```
# extra code - shows how well correlated z1 is to t: LLE worked fine
plt.title("$z_1$ vs $t$")
plt.scatter(X_unrolled[:, 0], t, c=t, cmap=darker_hot)
plt.xlabel("$z_1$")
plt.ylabel("$t$", rotation=0)
plt.grid(True)
plt.show()
```



Lưu ý: Tôi đã thêm `normalized_stress=False` bên dưới để tránh một cảnh báo về việc giá trị mặc định cho siêu tham số đó sẽ thay đổi trong tương lai. Vui lòng xem [documentation](#) để biết thêm chi tiết.

```
from sklearn.manifold import MDS

mds = MDS(n_components=2, normalized_stress=False, random_state=42)
X_reduced_mds = mds.fit_transform(X_swiss)
```

```
from sklearn.manifold import Isomap

isomap = Isomap(n_components=2)
X_reduced_isomap = isomap.fit_transform(X_swiss)
```

```
from sklearn.manifold import TSNE

tsne = TSNE(n_components=2, init="random", learning_rate="auto",
            random_state=42)
X_reduced_tsne = tsne.fit_transform(X_swiss)
```

```
# extra code - this cell generates and saves Figure 8-11

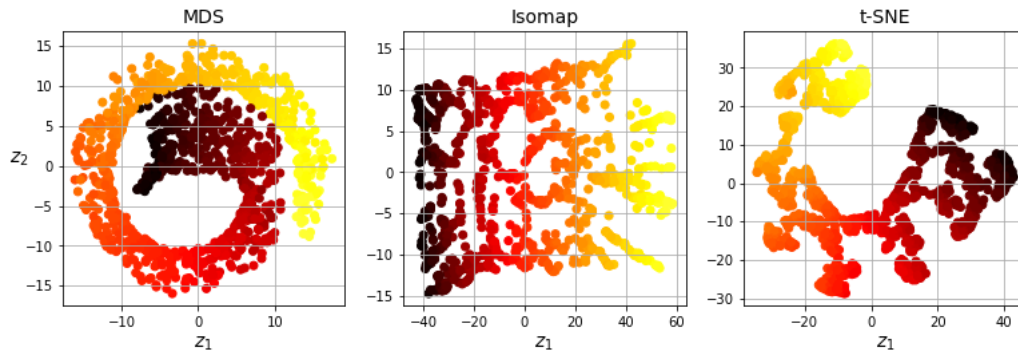
titles = ["MDS", "Isomap", "t-SNE"]

plt.figure(figsize=(11, 4))

for subplot, title, X_reduced in zip((131, 132, 133), titles,
                                     (X_reduced_mds, X_reduced_isomap, X_reduced_tsne)):
    plt.subplot(subplot)
    plt.title(title)
    plt.scatter(X_reduced[:, 0], X_reduced[:, 1], c=t, cmap=darker_hot)
    plt.xlabel("$z_1$")
    if subplot == 131:
```

```
plt.ylabel("$z_2$", rotation=0)
plt.grid(True)

save_fig("other_dim_reduction_plot")
plt.show()
```



▼ Tài liệu bổ sung – Kernel PCA

```
from sklearn.decomposition import KernelPCA

rbf_pca = KernelPCA(n_components=2, kernel="rbf", gamma=0.04, random_state=42)
X_reduced = rbf_pca.fit_transform(X_swiss)

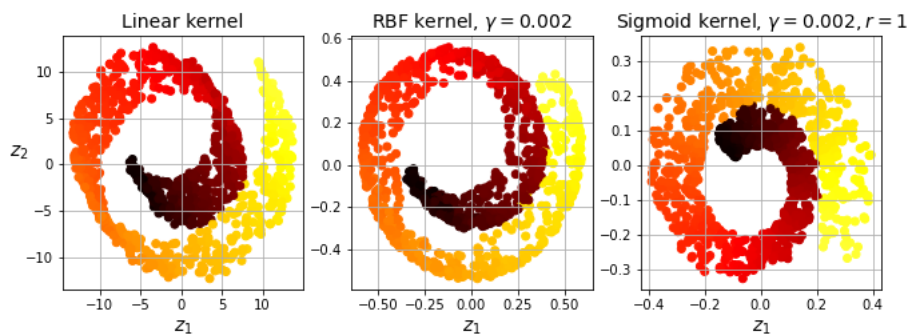
lin_pca = KernelPCA(kernel="linear")
rbf_pca = KernelPCA(kernel="rbf", gamma=0.002)
sig_pca = KernelPCA(kernel="sigmoid", gamma=0.002, coef0=1)

kernel_pcas = ((lin_pca, "Linear kernel"),
                (rbf_pca, rf"RBF kernel, $\gamma$={rbf_pca.gamma}$"),
                (sig_pca, rf"Sigmoid kernel, $\gamma$={sig_pca.gamma}, r={sig_pca.coef0}$"))

plt.figure(figsize=(11, 3.5))
for idx, (kpca, title) in enumerate(kernel_pcas):
    kpca.n_components = 2
    kpca.random_state = 42
    X_reduced = kpca.fit_transform(X_swiss)

    plt.subplot(1, 3, idx + 1)
    plt.title(title)
    plt.scatter(X_reduced[:, 0], X_reduced[:, 1], c=t, cmap=darker_hot)
    plt.xlabel("$z_1$")
    if idx == 0:
        plt.ylabel("$z_2$", rotation=0)
    plt.grid()

plt.show()
```



▼ Gợi ý giải bài tập

✓ 1. tới 8.

1. Các động lực chính cho việc giảm chiều dữ liệu (dimensionality reduction) là:

- Để tăng tốc thuật toán huấn luyện tiếp theo (trong một số trường hợp, nó thậm chí có thể loại bỏ nhiễu và các đặc trưng dư thừa, làm cho thuật toán huấn luyện hoạt động tốt hơn)
- Để trực quan hóa dữ liệu và hiểu rõ hơn về các đặc trưng quan trọng nhất
- Để tiết kiệm không gian (nén)

Các nhược điểm chính là:

- Một số thông tin bị mất, có khả năng làm giảm hiệu suất của các thuật toán huấn luyện tiếp theo.
 - Nó có thể tốn nhiều tính toán.
 - Nó làm tăng thêm một chút độ phức tạp cho các quy trình Học máy của bạn.
 - Các đặc trưng được chuyển đổi thường khó diễn giải.
2. Lời nguyền của chiều dữ liệu (The curse of dimensionality) đề cập đến thực tế là nhiều vấn đề không tồn tại trong không gian chiều thấp lại nảy sinh trong không gian chiều cao. Trong Học máy, một biểu hiện phổ biến là thực tế các vectơ chiều cao được lấy mẫu ngẫu nhiên thường cách xa nhau, làm tăng nguy cơ quá khớp (overfitting) và khiến việc xác định các mẫu trở nên rất khó khăn nếu không có nhiều dữ liệu huấn luyện.
3. Khi chiều dữ liệu của một tập dữ liệu đã được giảm bằng một trong các thuật toán chúng ta đã thảo luận, hầu như luôn là không thể đảo ngược hoàn hảo hoạt động, bởi vì một số thông tin bị mất trong quá trình giảm chiều dữ liệu. Hơn nữa, trong khi một số thuật toán (chẳng hạn như PCA) có quy trình biến đổi ngược đơn giản có thể tái tạo lại một tập dữ liệu tương đối giống với bản gốc, các thuật toán khác (chẳng hạn như t-SNE) thì không.
4. PCA có thể được sử dụng để giảm đáng kể chiều dữ liệu của hầu hết các tập dữ liệu, ngay cả khi chúng là phi tuyến tính cao, bởi vì nó ít nhất có thể loại bỏ các chiều vô dụng. Tuy nhiên, nếu không có chiều vô dụng—như trong tập dữ liệu cuộn Thụy Sĩ (Swiss roll)—thì việc giảm chiều dữ liệu bằng PCA sẽ làm mất quá nhiều thông tin. Bạn muốn mở cuộn Thụy Sĩ ra, chứ không phải làm bẹp nó.
5. Đó là một câu hỏi mẹo: nó phụ thuộc vào tập dữ liệu. Hãy xem xét hai ví dụ cực đoan. Thứ nhất, giả sử tập dữ liệu bao gồm các điểm gần như thẳng hàng hoàn hảo. Trong trường hợp này, PCA có thể giảm tập dữ liệu xuống chỉ còn một chiều mà vẫn giữ được 95% phương sai. Bây giờ hãy tưởng tượng rằng tập dữ liệu bao gồm các điểm hoàn toàn ngẫu nhiên, phân tán khắp 1.000 chiều. Trong trường hợp này cần khoảng 950 chiều để giữ lại 95% phương sai. Vì vậy, câu trả lời là, nó phụ thuộc vào tập dữ liệu, và có thể là bất kỳ số nào giữa 1 và 950. Vẽ đồ thị phương sai được giải thích dưới dạng hàm của số chiều là một cách để có được ý tưởng sơ bộ về chiều nội tại của tập dữ liệu.
6. PCA thông thường (Regular PCA) là mặc định, nhưng nó chỉ hoạt động nếu tập dữ liệu nằm gọn trong bộ nhớ. PCA tăng dần (Incremental PCA) hữu ích cho các tập dữ liệu lớn không nằm gọn trong bộ nhớ, nhưng nó chậm hơn PCA thông thường, vì vậy nếu tập dữ liệu nằm gọn trong bộ nhớ bạn nên ưu tiên PCA thông thường. PCA tăng dần cũng hữu ích cho các tác vụ trực tuyến (online tasks), khi bạn cần áp dụng PCA ngay lập tức, mỗi khi một trường hợp mới đến. PCA ngẫu nhiên (Randomized PCA) hữu ích khi bạn muốn giảm đáng kể chiều dữ liệu và tập dữ liệu nằm gọn trong bộ nhớ; trong trường hợp này, nó nhanh hơn nhiều so với PCA thông thường. Cuối cùng, Phép chiếu ngẫu nhiên (Random Projection) rất tuyệt vời cho các tập dữ liệu có chiều dữ liệu rất cao.
7. Một cách trực quan, một thuật toán giảm chiều dữ liệu hoạt động tốt nếu nó loại bỏ nhiều chiều khỏi tập dữ liệu mà không làm mất quá nhiều thông tin. Một cách để đo lường điều này là áp dụng phép biến đổi ngược và đo lường lỗi tái tạo (reconstruction error). Tuy nhiên, không phải tất cả các thuật toán giảm chiều dữ liệu đều cung cấp phép biến đổi ngược. Ngoài ra, nếu bạn đang sử dụng giảm chiều dữ liệu làm bước tiền xử lý trước một thuật toán Học máy khác (ví dụ: bộ phân loại Rừng ngẫu nhiên), thì bạn chỉ cần đo lường hiệu suất của thuật toán thứ hai đó; nếu việc giảm chiều dữ liệu không làm mất quá nhiều thông tin, thì thuật toán sẽ hoạt động tốt như khi sử dụng tập dữ liệu gốc.
8. Hoàn toàn có lý khi xâu chuỗi hai thuật toán giảm chiều dữ liệu khác nhau. Một ví dụ phổ biến là sử dụng PCA hoặc Phép chiếu ngẫu nhiên để nhanh chóng loại bỏ một số lượng lớn các chiều vô dụng, sau đó áp dụng một thuật toán giảm chiều dữ liệu khác chậm hơn nhiều, chẳng hạn như LLE. Cách tiếp cận hai bước này có thể sẽ mang lại hiệu suất gần như tương tự như chỉ sử dụng LLE, nhưng trong một khoảng thời gian ngắn hơn nhiều.

✓ 9.

Bài tập: Tải tập dữ liệu MNIST (được giới thiệu trong chương 3) và chia nó thành tập huấn luyện và tập kiểm tra (lấy 60.000 trường hợp đầu tiên cho việc huấn luyện và 10.000 trường hợp còn lại cho việc kiểm tra).

Bộ dữ liệu MNIST đã được tải lên trước đó.

```
X_train = mnist.data[:60000]
y_train = mnist.target[:60000]

X_test = mnist.data[60000:]
y_test = mnist.target[60000:]
```

Bài tập: Huấn luyện một bộ phân loại Rừng ngẫu nhiên (Random Forest classifier) trên tập dữ liệu và tính thời gian cần thiết, sau đó đánh giá mô hình thu được trên tập kiểm tra.

```
rnd_clf = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
%time rnd_clf.fit(X_train, y_train)
```

```
CPU times: user 37.7 s, sys: 588 ms, total: 38.3 s
Wall time: 38.4 s
RandomForestClassifier(random_state=42)
```

```
from sklearn.metrics import accuracy_score

y_pred = rnd_clf.predict(X_test)
accuracy_score(y_test, y_pred)
```

```
0.9705
```

Bài tập: Tiếp theo, sử dụng PCA để giảm chiều dữ liệu của tập dữ liệu, với tỉ lệ phương sai được giải thích (explained variance ratio) là 95%.

```
from sklearn.decomposition import PCA

pca = PCA(n_components=0.95)
X_train_reduced = pca.fit_transform(X_train)
```

Bài tập: Huấn luyện một bộ phân loại Rừng ngẫu nhiên mới trên tập dữ liệu đã giảm và xem mất bao lâu. Việc huấn luyện có nhanh hơn nhiều không?

```
rnd_clf_with_pca = RandomForestClassifier(n_estimators=100, random_state=42)
%time rnd_clf_with_pca.fit(X_train_reduced, y_train)
```

```
CPU times: user 1min 28s, sys: 590 ms, total: 1min 28s
Wall time: 1min 28s
RandomForestClassifier(random_state=42)
```

Ồi không! Việc huấn luyện thực sự chậm hơn khoảng hai lần bây giờ! Tại sao lại như vậy? Chà, như chúng ta đã thấy trong chương này, giảm chiều dữ liệu không phải lúc nào cũng dẫn đến thời gian huấn luyện nhanh hơn: nó phụ thuộc vào tập dữ liệu, mô hình và thuật toán huấn luyện. Xem hình 8-6 (các đồ thị `manifold_decision_boundary_plot*` ở trên). Nếu bạn thử `SGDClassifier` thay vì `RandomForestClassifier`, bạn sẽ thấy thời gian huấn luyện giảm đi 3 lần khi sử dụng PCA. Thật ra, chúng ta sẽ làm điều này ngay bây giờ, nhưng trước tiên hãy kiểm tra độ chính xác của bộ phân loại rừng ngẫu nhiên mới.

Bài tập: Tiếp theo, đánh giá bộ phân loại trên tập kiểm tra: nó so sánh như thế nào với bộ phân loại trước?

```
X_test_reduced = pca.transform(X_test)

y_pred = rnd_clf_with_pca.predict(X_test_reduced)
accuracy_score(y_test, y_pred)
```

```
0.9481
```

Thường thì hiệu suất có thể giảm nhẹ khi giảm số chiều, bởi vì chúng ta mất đi một số tín hiệu có thể hữu ích trong quá trình này. Tuy nhiên, sự giảm hiệu suất trong trường hợp này là khá nghiêm trọng. Vậy nên PCA thực sự không hữu ích: nó làm chậm quá trình đào tạo và giảm hiệu suất. 😞

Bài tập: Thử lại với một `SGDClassifier`. PCA giúp ích được bao nhiêu bây giờ?

```
from sklearn.linear_model import SGDClassifier
```



```
sgd_clf = SGDClassifier(random_state=42)
%time sgd_clf.fit(X_train, y_train)
```

```
CPU times: user 2min 34s, sys: 906 ms, total: 2min 35s
Wall time: 2min 35s
SGDClassifier(random_state=42)
```

```
y_pred = sgd_clf.predict(X_test)
accuracy_score(y_test, y_pred)
```

```
0.874
```

Được rồi, vậy là `SGDClassifier` mất nhiều thời gian để huấn luyện trên tập dữ liệu này hơn `RandomForestClassifier`, cộng thêm nó hoạt động kém hơn trên tập kiểm tra. Nhưng đó không phải là điều chúng ta quan tâm lúc này, chúng ta muốn xem PCA có thể giúp `SGDClassifier` được bao nhiêu. Hãy huấn luyện nó bằng cách sử dụng tập dữ liệu đã giảm:

```
sgd_clf_with_pca = SGDClassifier(random_state=42)
%time sgd_clf_with_pca.fit(X_train_reduced, y_train)
```

```
CPU times: user 29.7 s, sys: 418 ms, total: 30.2 s
Wall time: 27.9 s
SGDClassifier(random_state=42)
```

Phải! Giảm chiều dữ liệu đã dẫn đến tăng tốc độ khoảng 5 lần. :) Hãy kiểm tra độ chính xác của mô hình:

```
y_pred = sgd_clf_with_pca.predict(X_test_reduced)
accuracy_score(y_test, y_pred)
```

```
0.8959
```

Rất tuyệt! PCA không chỉ mang lại cho chúng tôi một tốc độ tăng 5 lần, mà còn cải thiện hiệu suất một chút.

Vậy là bạn đã có điều này: PCA có thể mang lại cho bạn sự tăng tốc đáng kể, và nếu bạn may mắn, một sự cải thiện về hiệu suất... nhưng điều đó thực sự không đảm bảo: nó phụ thuộc vào mô hình và tập dữ liệu!

10.

Bài tập: Sử dụng *t-SNE* để giảm 5.000 hình ảnh đầu tiên của tập dữ liệu MNIST xuống hai chiều và vẽ kết quả bằng *Matplotlib*. Bạn có thể sử dụng biểu đồ phân tán (*scatterplot*) với 10 màu khác nhau để đại diện cho lớp mục tiêu của mỗi hình ảnh.

Hãy giới hạn chúng ta trong 5.000 hình ảnh đầu tiên của tập huấn luyện MNIST để tăng tốc độ rất nhiều.

```
X_sample, y_sample = X_train[:5000], y_train[:5000]
```

Hãy sử dụng *t-SNE* để giảm chiều dữ liệu xuống 2D để chúng ta có thể vẽ biểu đồ của tập dữ liệu.

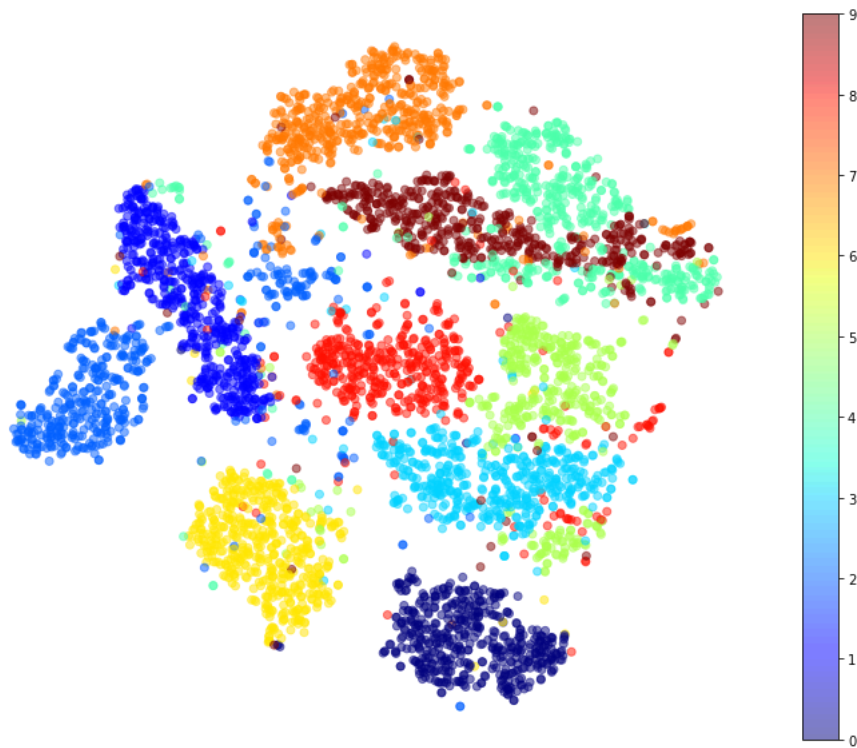
```
from sklearn.manifold import TSNE

tsne = TSNE(n_components=2, init="random", learning_rate="auto",
            random_state=42)
%time X_reduced = tsne.fit_transform(X_sample)
```

```
CPU times: user 5min 49s, sys: 31 s, total: 6min 20s
Wall time: 29.4 s
```

Bây giờ hãy sử dụng hàm `scatter()` của *Matplotlib* để vẽ một biểu đồ phân tán, sử dụng màu khác nhau cho mỗi chữ số.

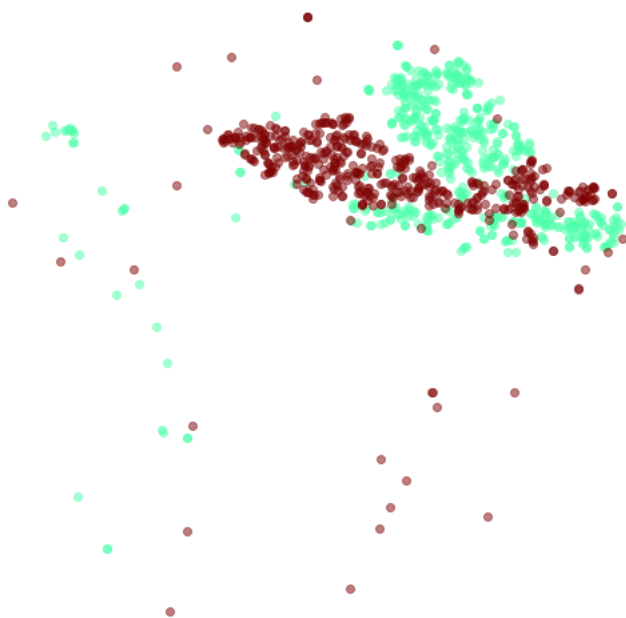
```
plt.figure(figsize=(13, 10))
plt.scatter(X_reduced[:, 0], X_reduced[:, 1],
            c=y_sample.astype(np.int8), cmap="jet", alpha=0.5)
plt.axis('off')
plt.colorbar()
plt.show()
```



Chẳng phải điều này thật đẹp sao? :) Hầu hết các chữ số được phân tách một cách rõ ràng khỏi những chữ số khác, mặc dù t-SNE không được cung cấp các mục tiêu: nó chỉ xác định các cụm hình ảnh tương tự. Nhưng vẫn còn một chút chồng chéo. Chẳng hạn, các số 3 và 5 chồng chéo nhau rất nhiều (ở phía bên phải của biểu đồ), và các số 4 và 9 cũng vậy (ở góc trên bên phải).

Chúng ta hãy tập trung vào chỉ hai chữ số 4 và 9:

```
plt.figure(figsize=(9, 9))
cmap = plt.cm.jet
for digit in ('4', '9'):
    plt.scatter(X_reduced[y_sample == digit, 0], X_reduced[y_sample == digit, 1],
                c=[cmap(float(digit) / 9)], alpha=0.5)
plt.axis('off')
plt.show()
```



Hãy xem liệu chúng ta có thể tạo ra một hình ảnh đẹp hơn bằng cách chạy t-SNE chỉ với 2 chữ số này:

```
idx = (y_sample == '4') | (y_sample == '9')
X_subset = X_sample[idx]
y_subset = y_sample[idx]

tsne_subset = TSNE(n_components=2, init="random", learning_rate="auto",
                  random_state=42)
X_subset_reduced = tsne_subset.fit_transform(X_subset)
```

```
plt.figure(figsize=(9, 9))
for digit in ('4', '9'):
    plt.scatter(X_subset_reduced[y_subset == digit, 0],
               X_subset_reduced[y_subset == digit, 1],
               c=[cmap(float(digit) / 9)], alpha=0.5)
plt.axis('off')
plt.show()
```



Điều đó tốt hơn nhiều, mặc dù vẫn còn một chút chồng chéo. Có lẽ một số số 4 thực sự trông giống số 9, và ngược lại. Sẽ rất tuyệt nếu chúng ta có thể trực quan hóa một vài chữ số từ mỗi vùng của biểu đồ này, để hiểu chuyện gì đang xảy ra. Trên thực tế, hãy làm điều đó ngay bây giờ.

Bài tập: Ngoài ra, bạn có thể thay thế mỗi chấm trong biểu đồ phân tán bằng lớp của trường hợp tương ứng (một chữ số từ 0 đến 9), hoặc thậm chí vẽ các phiên bản hình ảnh chữ số đã được thu nhỏ (nếu bạn vẽ tất cả các chữ số, hình ảnh trực quan sẽ quá lộn xộn, vì vậy bạn nên lấy mẫu ngẫu nhiên hoặc chỉ vẽ một trường hợp nếu không có trường hợp nào khác đã được vẽ ở khoảng cách gần). Bạn sẽ nhận được một hình ảnh trực quan đẹp với các cụm chữ số được tách biệt rõ ràng.

Hãy tạo một hàm `plot_digits()` sẽ vẽ một biểu đồ phân tán (tương tự như các biểu đồ phân tán ở trên) cộng với viết các chữ số có màu, với khoảng cách tối thiểu được đảm bảo giữa các chữ số này. Nếu các hình ảnh chữ số được cung cấp, chúng sẽ được vẽ thay thế. Việc triển khai này được lấy cảm hứng từ một trong những ví dụ tuyệt vời của Scikit-Learn ([plot_lle_digits](#), dựa trên một tập dữ liệu chữ số khác).

```
from sklearn.preprocessing import MinMaxScaler
from matplotlib.offsetbox import AnnotationBbox, OffsetImage

def plot_digits(X, y, min_distance=0.04, images=None, figsize=(13, 10)):
    # Let's scale the input features so that they range from 0 to 1
    X_normalized = MinMaxScaler().fit_transform(X)
    # Now we create the list of coordinates of the digits plotted so far.
    # We pretend that one is already plotted far away at the start, to
    # avoid `if` statements in the loop below
```

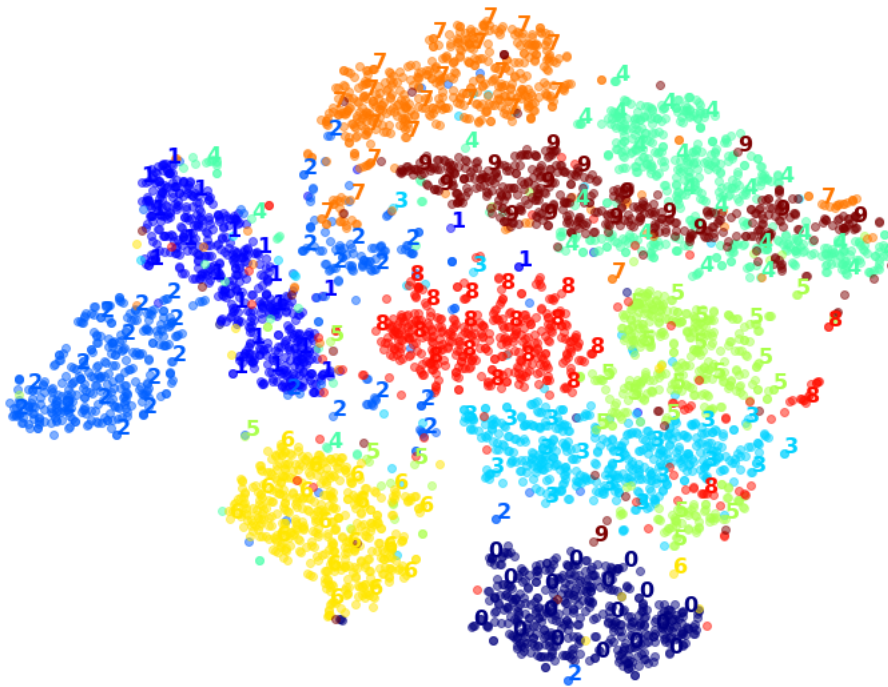
```

neighbors = np.array([[10., 10.]])
# The rest should be self-explanatory
plt.figure(figsize=figsize)
cmap = plt.cm.jet
digits = np.unique(y)
for digit in digits:
    plt.scatter(X_normalized[y == digit, 0], X_normalized[y == digit, 1],
                c=[cmap(float(digit) / 9)], alpha=0.5)
plt.axis("off")
ax = plt.gca() # get current axes
for index, image_coord in enumerate(X_normalized):
    closest_distance = np.linalg.norm(neighbors - image_coord, axis=1).min()
    if closest_distance > min_distance:
        neighbors = np.r_[neighbors, [image_coord]]
        if images is None:
            plt.text(image_coord[0], image_coord[1], str(int(y[index])),
                    color=cmap(float(y[index]) / 9),
                    fontdict={"weight": "bold", "size": 16})
        else:
            image = images[index].reshape(28, 28)
            imagebox = AnnotationBbox(OffsetImage(image, cmap="binary"),
                                     image_coord)
            ax.add_artist(imagebox)

```

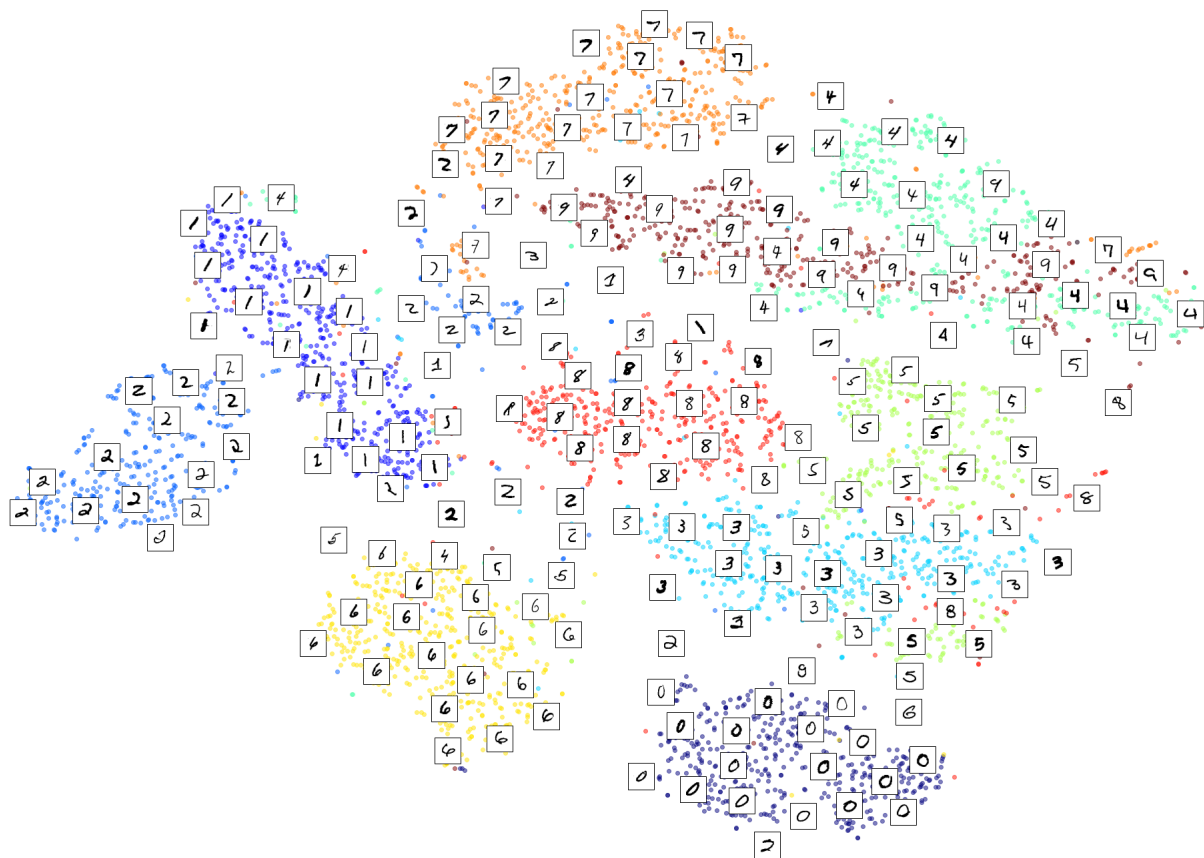
Hãy thử điều đó! Đầu tiên, chúng ta hãy hiển thị các chữ số có màu (không phải hình ảnh) cho tất cả 5.000 hình ảnh.

```
plot_digits(X_reduced, y_sample)
```



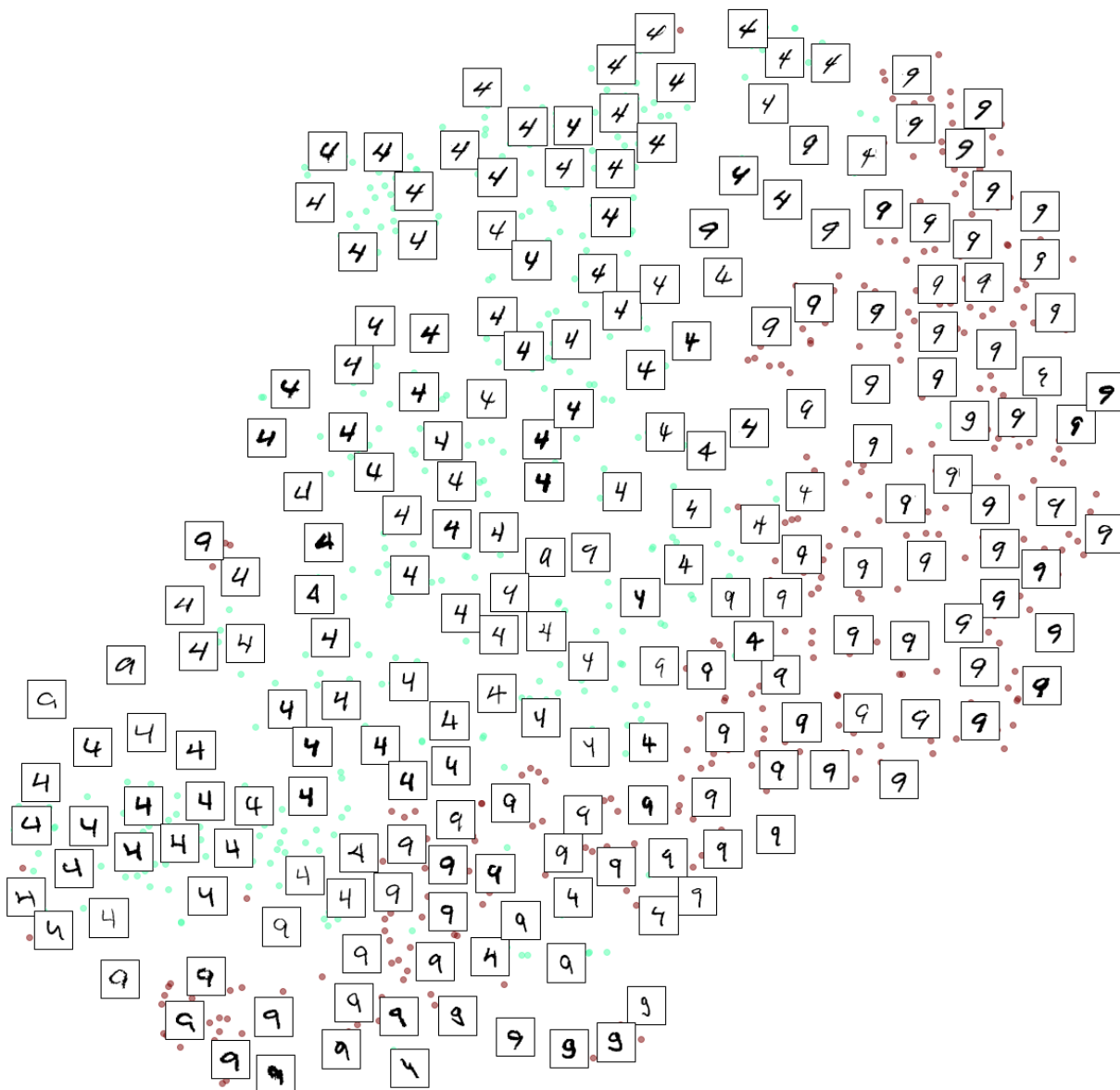
Chà, được rồi, nhưng không đẹp lắm. Hãy thử với các hình ảnh chữ số:

```
plot_digits(X_reduced, y_sample, images=X_sample, figsize=(35, 25))
```



Đẹp đó! Bây giờ hãy tập trung vào chỉ các chữ số 3 và 5:

```
plot_digits(X_subset_reduced, y_subset, images=X_subset, figsize=(22, 22))
```



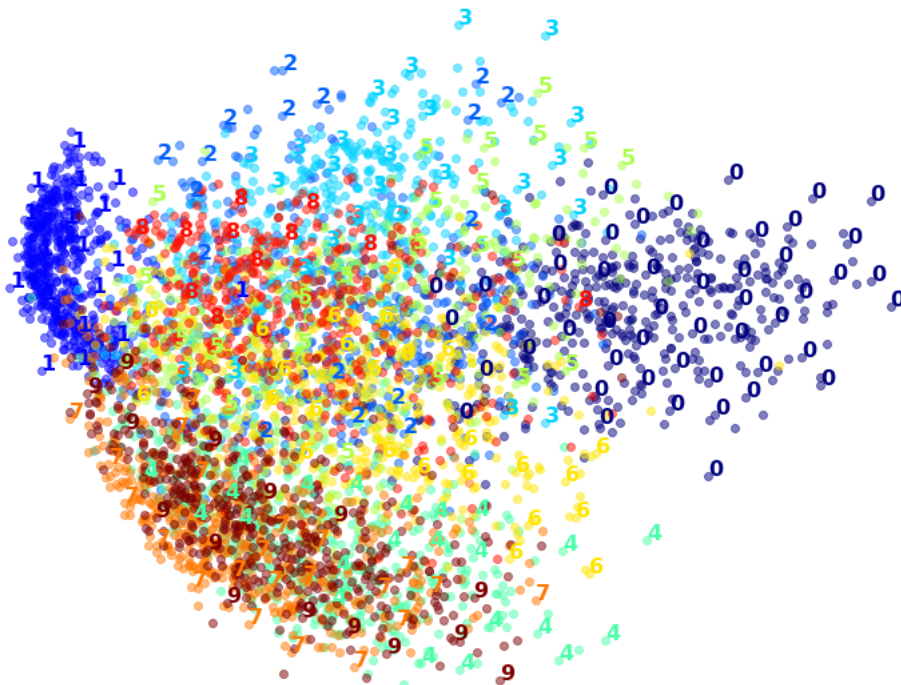
Lưu ý cách các chữ số 4 trông tương tự nhau được nhóm lại với nhau. Ví dụ, các chữ số 4 ngày càng nghiêng hơn khi chúng tiến gần đến đỉnh của hình vẽ. Các chữ số 9 nghiêng cũng ở gần đỉnh hơn. Một số chữ số 4 thực sự trông giống số 9, và ngược lại.

Bài tập: Thử sử dụng các thuật toán giảm chiều dữ liệu khác như PCA, LLE hoặc MDS và so sánh các hình ảnh trực quan thu được.

Hãy bắt đầu với PCA. Chúng ta cũng sẽ tính thời gian cần thiết:

```
pca = PCA(n_components=2, random_state=42)
%time X_pca_reduced = pca.fit_transform(X_sample)
plot_digits(X_pca_reduced, y_sample)
plt.show()
```

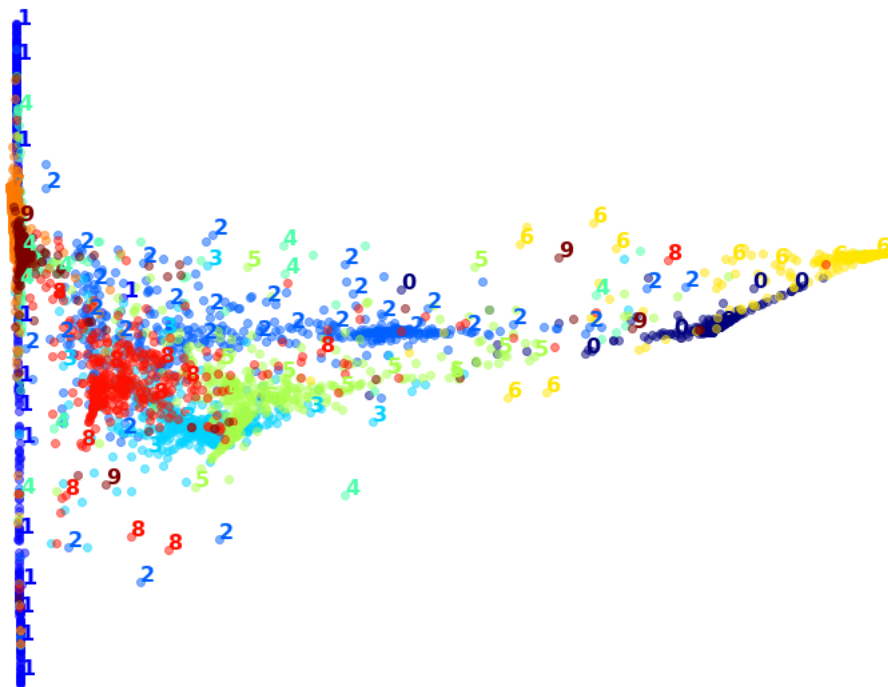
CPU times: user 1.49 s, sys: 219 ms, total: 1.71 s
Wall time: 157 ms



Chà, PCA cực kỳ nhanh! Nhưng mặc dù chúng ta thấy một vài cụm, có quá nhiều sự chồng chéo. Hãy thử LLE:

```
lle = LocallyLinearEmbedding(n_components=2, random_state=42)
%time X_lle_reduced = lle.fit_transform(X_sample)
plot_digits(X_lle_reduced, y_sample)
plt.show()
```

CPU times: user 1min 16s, sys: 7.77 s, total: 1min 24s
Wall time: 8.86 s

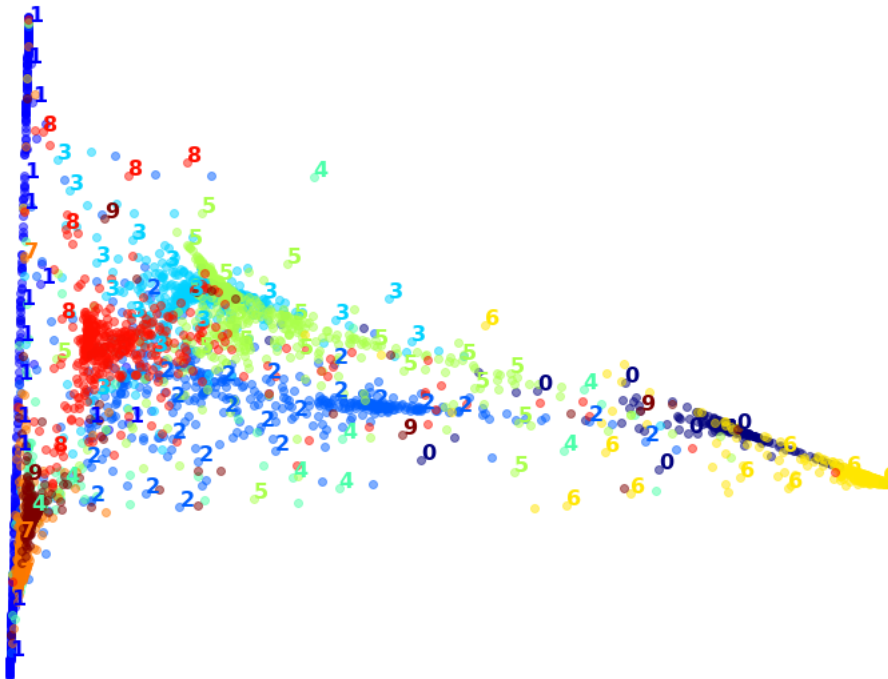


Việc đó tốn nhiều thời gian hơn, nhưng kết quả lại không hề tốt. Hãy xem điều gì xảy ra nếu chúng ta áp dụng PCA trước, giữ lại 95% phương sai:

```
pca_lle = make_pipeline(PCA(n_components=0.95),  
                        LocallyLinearEmbedding(n_components=2, random_state=42))
```

```
%time X_pca_lle_reduced = pca_lle.fit_transform(X_sample)  
plot_digits(X_pca_lle_reduced, y_sample)  
plt.show()
```

CPU times: user 1min 18s, sys: 6.54 s, total: 1min 24s
Wall time: 7.57 s



Kết quả cũng tệ tương đương, nhưng lần này việc huấn luyện nhanh hơn một chút.

Hãy dùng MDS:

Cảnh báo, ô sau có thể mất khoảng 10-30 phút để chạy, tùy thuộc vào phần cứng của bạn:

```
%time X_mds_reduced = MDS(n_components=2, normalized_stress=False, random_state=42).fit_transform(X_sample)  
plot_digits(X_mds_reduced, y_sample)  
plt.show()
```



```
CPU times: user 1h 5min 8s, sys: 7min 56s, total: 1h 13min 4s
Wall time: 12min 12s
```

Meh. Điều này trông không được đẹp lắm, tất cả các cụm bị chồng chéo quá nhiều. Hãy thử với PCA trước, có lẽ sẽ nhanh hơn?

Cảnh báo, ô này có thể mất khoảng 10-30 phút để chạy, tùy thuộc vào phần cứng của bạn:

```
pca_mds = make_pipeline(
    PCA(n_components=0.95, random_state=42),
    MDS(n_components=2, normalized_stress=False, random_state=42)
)
```

```
%time X_pca_mds_reduced = pca_mds.fit_transform(X_sample)
plot_digits(X_pca_mds_reduced, y_sample)
plt.show()
```

```
CPU times: user 1h 14min 58s, sys: 5min 56s, total: 1h 20min 4s
Wall time: 12min 12s
```